Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №1
по дисциплине «Вычислительная математика»

Вариант: **19**

Преподаватель:
Малышева Татьяна Алексеевна

Выполнил: Якименко Владислав Игоревич
Группа: P3213

Санкт-Петербург, 2026 г

Цель работы
Изучить и реализовать решение системы линейных алгебраических уравнений методом Гаусса с выбором главного элемента по столбцу (вариант 19)

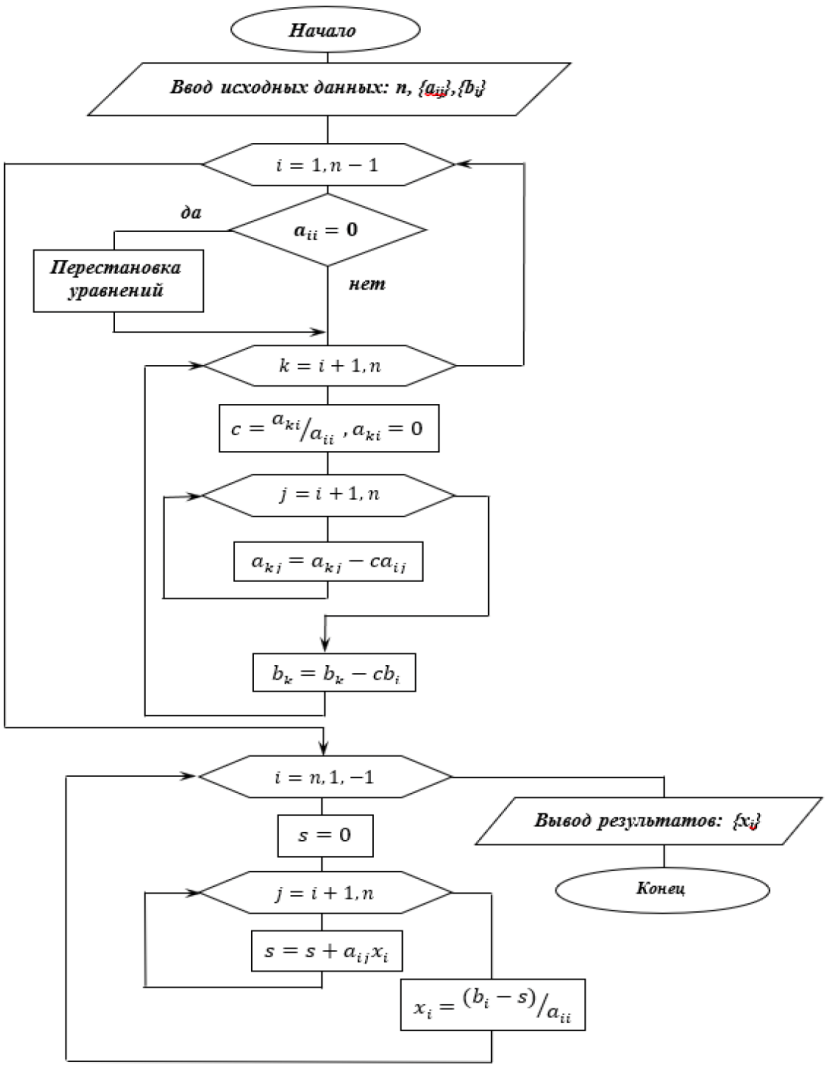
Описание метода
Метод Гаусса с выбором главного элемента по столбцу решает СЛАУ $A_x = b$ в два этапа:

1. **Прямой ход**: для каждого столбца выбирается максимальный по модулю элемент, при необходимости меняются строки, затем зануляются элементы под диагональю. В итоге получаем верхнетреугольную матрицу
2. **Обратный ход**: начиная с последней строки, последовательно находим неизвестные x_n, x_{n-1}, …,x_1

Определитель вычисляется как произведение диагональных элементов треугольной матрицы с учетом знака перестановок строк.

Корректность решения проверяется по вектору невязок $r = A_x - b$

В программе вычисления выполняются в точной рациональной арифметике (BigInt), что снижает ошибки округления.



Код программы

https://github.com/CompMathLabs/2026-p3212-lab1-delovoyhomie

gaussianColumnPivot.js

```
'use strict';

const { Fraction } = require('./fraction');

function cloneFraction(value) {
  return new Fraction(value.numerator, value.denominator);
}

function validateSystem(matrix, vector) {
  if (!Array.isArray(matrix) || matrix.length === 0) {
    throw new Error('Матрица коэффициентов должна быть непустым массивом.');
  }

  const n = matrix.length;
  if (n > 20) {
    throw new Error('Размерность n должна быть не больше 20.');
  }

  if (!Array.isArray(vector) || vector.length !== n) {
    throw new Error('Размер вектора B должен совпадать с количеством строк матрицы A.');
  }

  for (let row = 0; row < n; row += 1) {
    if (!Array.isArray(matrix[row]) || matrix[row].length !== n) {
      throw new Error(`Строка ${row + 1} матрицы A должна содержать ${n} элементов.`);
    }
  }
}

function calculateRanks(augmentedMatrix, n) {
  let rankA = 0;
  let rankAugmented = 0;

  for (let row = 0; row < n; row += 1) {
    let hasA = false;
    for (let col = 0; col < n; col += 1) {
      if (!augmentedMatrix[row][col].isZero()) {
        hasA = true;
        break;
      }
    }

    const hasB = !augmentedMatrix[row][n].isZero();
    if (hasA) {
      rankA += 1;
    }
    if (hasA || hasB) {
      rankAugmented += 1;
    }
  }

  return { rankA, rankAugmented };
}

function computeResidualVector(originalMatrix, originalVector, solution) {
  const n = originalMatrix.length;
  const residual = [];

  for (let i = 0; i < n; i += 1) {
```

```
    let sum = Fraction.zero();
    for (let j = 0; j < n; j += 1) {
      sum = sum.add(originalMatrix[i][j].mul(solution[j]));
    }
    residual.push(sum.sub(originalVector[i]));
  }

  return residual;
}

class GaussianColumnPivotSolver {
  solve(matrix, vector) {
    validateSystem(matrix, vector);

    const n = matrix.length;
    const augmented = matrix.map((row, rowIndex) => {
      const cloned = row.map((value) => cloneFraction(value));
      cloned.push(cloneFraction(vector[rowIndex]));
      return cloned;
    });

    let swapCount = 0;

    for (let column = 0; column < n; column += 1) {
      let pivotRow = column;
      let maxAbsValue = augmented[column][column].abs();

      for (let row = column + 1; row < n; row += 1) {
        const candidate = augmented[row][column].abs();
        if (candidate.compare(maxAbsValue) > 0) {
          maxAbsValue = candidate;
          pivotRow = row;
        }
      }

      if (maxAbsValue.isZero()) {
        continue;
      }

      if (pivotRow !== column) {
        [augmented[column], augmented[pivotRow]] = [augmented[pivotRow],
augmented[column]];
        swapCount += 1;
      }

      for (let row = column + 1; row < n; row += 1) {
        if (augmented[row][column].isZero()) {
          continue;
        }

        const factor = augmented[row][column].div(augmented[column][column]);
        for (let col = column; col <= n; col += 1) {
          augmented[row][col] =
augmented[row][col].sub(factor.mul(augmented[column][col]));
        }
      }
    }

    const triangularMatrix = augmented.map((row) => row.map((value) =>
cloneFraction(value)));
    const ranks = calculateRanks(augmented, n);

    let status = 'unique';
    let message = 'Единственное решение найдено.';

    if (ranks.rankA < ranks.rankAugmented) {
      status = 'no_solution';
      message = 'Система несовместна: решений нет.';
    } else if (ranks.rankA < n) {
```

```
      status = 'infinite';
      message = 'Система совместна, но имеет бесконечно много решений.';
    }

    let determinant = Fraction.one();
    if (ranks.rankA < n) {
      determinant = Fraction.zero();
    } else {
      for (let i = 0; i < n; i += 1) {
        determinant = determinant.mul(augmented[i][i]);
      }
      if (swapCount % 2 === 1) {
        determinant = determinant.neg();
      }
    }

    let solution = null;
    let residualVector = null;

    if (status === 'unique') {
      solution = new Array(n);

      for (let row = n - 1; row >= 0; row -= 1) {
        let sum = Fraction.zero();
        for (let col = row + 1; col < n; col += 1) {
          sum = sum.add(augmented[row][col].mul(solution[col]));
        }

        const diagonal = augmented[row][row];
        if (diagonal.isZero()) {
          throw new Error('Нулевой элемент на диагонали при обратном ходе.');
        }

        solution[row] = augmented[row][n].sub(sum).div(diagonal);
      }

      residualVector = computeResidualVector(matrix, vector, solution);
    }

    return {
      status,
      message,
      triangularMatrix,
      solution,
      residualVector,
      determinant,
      swapCount,
      rankA: ranks.rankA,
      rankAugmented: ranks.rankAugmented
    };
  }
}

module.exports = {
  GaussianColumnPivotSolver
};
```

Примеры работы команды:

2 - ввод из файла

Источник данных:
1 - ввод с клавиатуры
2 - ввод из файла
3 - случайная матрица заданной размерности
Выберите режим (1/2/3): 2
Введите путь к файлу: examples/test1.txt

==========
БЛОК 1. НАШИ ВЫЧИСЛЕНИЯ
Источник: собственный метод Гаусса + точные дроби Fraction(BigInt)
==========

Треугольная матрица (с преобразованным столбцом B):
3 0.5 2	11.5
0 1.75 -2	-7.75
0 0 7.357142857142857143	22.071428571428571429

Определитель (из треугольной матрицы):
det(A) = -38.625
Перестановок строк: 1
rank(A) = 3, rank([A|B]) = 3

Статус решения:
Единственное решение найдено.

Вектор неизвестных:
x1 = 2
x2 = -1
x3 = 3

Вектор невязок r = A*x - B:
r1 = 0
r2 = 0
r3 = 0

==========
БЛОК 2. ВЫЧИСЛЕНИЯ БИБЛИОТЕКИ
Источник: mathjs (числа с плавающей точкой Number/float64)
==========
det_lib(A) = -38.62500000000000

Вектор неизвестных (библиотека):
x_lib1 = 2.000000000000000
x_lib2 = -1.000000000000000
x_lib3 = 3.000000000000000

Вектор невязок библиотеки r_lib = A*x_lib - B:
r_lib1 = 0.000000000000000
r_lib2 = 0.000000000000000
r_lib3 = 0.000000000000000

==========
БЛОК 3. СРАВНЕНИЕ НАШЕГО МЕТОДА И БИБЛИОТЕКИ

Источник: расчёт абсолютных отклонений между двумя полученными решениями
==========
|det(наш) - det(библ)| = 0.000000000000000

Абсолютные отклонения по компонентам |x - x_lib|:
dx1 = 0.000000000000000
dx2 = 0.000000000000000
dx3 = 0.000000000000000
max |x - x_lib| = 0.000000000000000

3 - случайная матрица заданной размерности
Источник данных:
1 - ввод с клавиатуры
2 - ввод из файла
3 - случайная матрица заданной размерности
Выберите режим (1/2/3): 3
Введите размерность n (1..20): 5
Минимальное целое значение: 1
Максимальное целое значение: 5

==========
БЛОК 1. НАШИ ВЫЧИСЛЕНИЯ
Источник: собственный метод Гаусса + точные дроби Fraction(BigInt)
==========

Треугольная матрица (с преобразованным столбцом B):
5 5 4 5 4	3
0 1 2.6 0 -1.4	2.2
0 0 3.4 2 1.4	0.8
0 0 0 2.705882352941176471 3.294117647058823529	-0.117647058823529412
0 0 0 0 2.043478260869565217	-0.608695652173913043

Определитель (из треугольной матрицы):
det(A) = -94
Перестановок строк: 3
rank(A) = 5, rank([A|B]) = 5

Статус решения:
Единственное решение найдено.

Вектор неизвестных:
x1 = -0.9574468085106382978723404255
x2 = 1.3404255319148936170212765957
x3 = 0.1702127659574468085106382979
x4 = 0.3191489361702127659574468085
x5 = -0.2978723404255319148936170213

Вектор невязок r = A*x - B:
r1 = 0
r2 = 0
r3 = 0
r4 = 0
r5 = 0

==========
БЛОК 2. ВЫЧИСЛЕНИЯ БИБЛИОТЕКИ
Источник: mathjs (числа с плавающей точкой Number/float64)
==========
det_lib(A) = -94.00000000000000

Вектор неизвестных (библиотека):
x_lib1 = -0.9574468085106383
x_lib2 = 1.340425531914894
x_lib3 = 0.1702127659574468
x_lib4 = 0.3191489361702128
x_lib5 = -0.2978723404255320

Вектор невязок библиотеки r_lib = A*x_lib - B:
r_lib1 = 0.000000000000000
r_lib2 = 4.440892098500626e-16
r_lib3 = -4.440892098500626e-16
r_lib4 = -8.881784197001252e-16
r_lib5 = 0.000000000000000

==========
БЛОК 3. СРАВНЕНИЕ НАШЕГО МЕТОДА И БИБЛИОТЕКИ
Источник: расчёт абсолютных отклонений между двумя полученными решениями
==========
|det(наш) - det(библ)| = 0.000000000000000

Абсолютные отклонения по компонентам |x - x_lib|:
dx1 = 0.000000000000000
dx2 = 0.000000000000000
dx3 = 2.775557561562891e-17
dx4 = 5.551115123125783e-17
dx5 = 1.110223024625157e-16
max |x - x_lib| = 1.110223024625157e-16

Вывод:
В результате выполнения данной лабораторной работой я научился работать с методом Гаусса с выбором главного элемента по столбцу для СЛАУ, реализовав программу на языке программирования JS, что по сравнению с обычным методом Гаусса повышает численную устойчивость и снижает влияние ошибок округления. Итерационные методы (простых итераций и Гаусса–Зейделя) строят последовательность приближений и эффективны на больших системах, но требуют условий сходимости и критерия остановки.